

Model Optimization & Deployment

Model Inferencing • Quantization

Part 1 — Model Inferencing

Inferencing is the process of running a trained model to generate outputs. In production, you want it to be fast, cheap, and scalable. The four main techniques to make LLM inferencing efficient are GPU memory management, KV caching, continuous batching, and speculative decoding.

1.1 GPU Usage Techniques

GPUs are essential for LLM inferencing because matrix multiplications at scale run orders of magnitude faster on GPU than CPU. But GPU memory (VRAM) is limited and expensive, so how you use it matters a lot.

Model Parallelism

When a model is too large to fit on a single GPU, you split it across multiple GPUs. There are two ways to do this:

- **Tensor Parallelism:** split individual weight matrices across GPUs. Each GPU handles part of every layer. Fast but requires high-bandwidth communication between GPUs on every forward pass.
- **Pipeline Parallelism:** split the model layer by layer across GPUs. GPU 1 handles layers 1-10, GPU 2 handles layers 11-20, etc. Less communication needed but GPUs can sit idle waiting for the previous one to finish (pipeline bubble).

Memory Optimization

- **Half-precision (FP16 / BF16):** store weights in 16-bit instead of 32-bit floats. Cuts VRAM usage in half with minimal accuracy loss. Almost always used in production.
- **Flash Attention:** recomputes attention scores on the fly instead of storing the full attention matrix. Saves massive VRAM for long sequences. Standard in most modern LLM serving frameworks.
- **Memory-mapped weights:** weights stay on disk, loaded into GPU on demand. Useful for extremely large models but slower.

Batching Requests

Sending multiple requests to the GPU at once (a batch) is far more efficient than sending them one by one, because GPU cores work in parallel. The question is how to batch smartly — which is where continuous batching comes in (covered below).

1.2 KV Cache

This is one of the most important concepts in LLM inference. Understanding it separates junior from senior AI engineers.

The problem it solves

LLMs generate text one token at a time (autoregressive generation). At each step, the attention mechanism needs to look at all previous tokens. Without caching, this means recomputing the Key and Value matrices for every previous token at every new step — extremely wasteful.

What KV Cache does

It stores the computed Key (K) and Value (V) matrices from the attention layers for all previous tokens. When generating the next token, you only compute K and V for the new token and look up the rest from the cache. This makes generation much faster.

The formula

KV Cache memory per token (for one layer):

```
Memory = 2 × num_heads × head_dim × bytes_per_element
```

The factor of 2 is for K and V (there's no Q cache because Q is always just the current token).

Full KV Cache size across all layers and all tokens in a sequence:

```
KV Cache size = 2 × num_layers × num_heads × head_dim × seq_len × bytes_per_element
```

Example for a 13B parameter model (FP16, 40 layers, 40 heads, head_dim=128):

```
= 2 × 40 × 40 × 128 × seq_len × 2 bytes  
= ~820 KB per token in the sequence  
For 4096 token context → ~3.3 GB just for KV cache
```

This is why long-context inference is so memory-hungry. KV cache grows linearly with sequence length.

KV Cache trade-offs

- **Pro:** dramatically speeds up generation — no recomputation for past tokens.
- **Con:** eats into VRAM. For long sequences or large batches, KV cache can use more memory than the model weights themselves.
- **Solution:** PagedAttention (used in vLLM) manages KV cache like OS virtual memory — allocates memory in pages on demand, avoids fragmentation, allows much larger batches.

1.3 Continuous Batching

Problem with static (naive) batching

In naive batching, you group N requests into a batch, run the whole batch until ALL requests are done, then start the next batch. The problem: requests have different lengths. Short ones finish early and the GPU sits idle waiting for long ones to finish. This wastes GPU capacity.

What continuous batching does

Also called iteration-level scheduling or in-flight batching. Instead of waiting for the whole batch to finish, as soon as one request completes, a new request immediately joins the batch for the next iteration. The batch is always full.

Static batching: Req A (5 tokens) + Req B (50 tokens) → both start together → GPU waits after step 5 for B to finish.

Continuous batching: Req A finishes at step 5 → Req C immediately slots in → GPU stays busy.

Why it matters

- GPU utilization goes from ~40-60% to close to 100%
- Throughput (requests per second) increases significantly
- Latency for individual short requests improves — they're not blocked by long ones
- Used by vLLM, TensorRT-LLM, and most modern serving frameworks

1.4 Speculative Decoding

The core problem

LLM generation is inherently sequential — you can't generate token N+1 until you have token N. This means you can't parallelize across time. GPU is underutilized because each token generation step is fast but you have to do them one by one.

The idea

Use a small, fast draft model to guess several tokens ahead, then verify all the guesses in one parallel pass through the large model.

Step by step

- **Step 1 — Draft:** A small model (e.g. 7B) generates K tokens speculatively, very fast.
- **Step 2 — Verify:** The large model (e.g. 70B) checks all K tokens in a single forward pass. It can do this in parallel because it's doing verification, not generation.
- **Step 3 — Accept/Reject:** Accept all tokens up to the first one the large model disagrees with. If all K are accepted, great — K tokens generated for the cost of ~1 large model pass. If first token is wrong, discard the rest and regenerate.

Why this works

The large model's forward pass over K tokens is barely more expensive than over 1 token, because the GPU processes all K positions in parallel. So if even a few tokens get accepted, you've made a net gain.

When it works best

- Tasks with predictable output (code completion, structured data)
- The draft and target model are from the same family (same tokenizer)
- The draft model is fast enough that drafting doesn't add significant overhead

Trade-offs

- **Pro:** 2-4x speedup in practice on suitable tasks without changing output quality.
 - **Con:** Adds complexity. Need to manage two models. Doesn't help much for highly creative or unpredictable outputs.
-

Part 2 — Quantization

Quantization means reducing the precision of model weights (and sometimes activations) from high-precision floats to lower-precision integers or smaller floats. The goal is to shrink model size and speed up inference, with minimal accuracy loss.

2.1 Why Quantization Matters

- **Model size:** A 7B parameter model in FP32 takes ~28 GB. In INT8 it's ~7 GB. In INT4 it's ~3.5 GB — fits on a single consumer GPU.
- **Speed:** Integer operations are faster than float operations on modern hardware. INT8 matrix multiplies can be 2-4x faster than FP16.
- **Cost:** Smaller models fit on smaller (cheaper) GPUs. Reduces deployment cost significantly.

The main trade-off is accuracy. The more you quantize, the more information you lose. The techniques below are all different ways to minimize that accuracy loss.

2.2 INT8 Quantization

Converts weights (and sometimes activations) from 32-bit or 16-bit floats to 8-bit integers.

How it works

Each floating point value is mapped to an integer in the range [-128, 127] using a scale factor and zero point:

```
quantized_value = round(float_value / scale) + zero_point
reconstructed = (quantized_value - zero_point) * scale
```

The scale is computed from the range of values in each weight tensor. Computation happens in INT8, then results are dequantized back to FP16/FP32 for output.

Variants

- **Post-Training Quantization (PTQ):** quantize after training, no retraining needed. Fast but some accuracy loss.
- **Quantization-Aware Training (QAT):** simulate quantization during training so the model learns to compensate. Better accuracy but needs retraining.

When to use INT8

- When you need moderate size reduction with minimal accuracy loss
 - INT8 is well-supported by hardware (NVIDIA Tensor Cores, TPUs)
 - Standard choice for production deployments where accuracy is important
-

2.3 INT4 Quantization

Pushes further — 4-bit integers, values only in range [-8, 7]. Even smaller model size but higher accuracy risk.

Challenges

4-bit has a very small range (only 16 possible values). Representing the full distribution of weights in just 16 values loses a lot of information. Naive INT4 gives bad accuracy.

How it's made practical

- **Group quantization:** instead of one scale per layer, use a separate scale for every small group of weights (e.g. 128 weights). This gives the quantizer more flexibility.
- **Mixed precision:** keep sensitive layers (first and last) in higher precision, quantize only the middle layers to INT4.
- **NF4 (NormalFloat4):** used in QLoRA — a 4-bit data type designed specifically for normally-distributed neural network weights. Better than pure INT4 for this purpose.

When to use INT4

- When the model must run on very constrained hardware (consumer GPUs, edge devices)
 - When combined with fine-tuning (QLoRA) to recover accuracy
 - Acceptable slight accuracy loss for major size/speed gains
-

2.4 GPTQ

GPTQ (Generative Pre-trained Transformer Quantization) is a post-training quantization method specifically designed for large language models, developed by researchers at ETH Zurich in 2022.

The core idea

Standard quantization quantizes each weight independently. GPTQ quantizes weights layer by layer, and uses the Hessian (second-order derivative information) to adjust remaining weights to compensate for the error introduced by quantizing earlier weights.

Algorithm (simplified)

- **Step 1:** Run a small calibration dataset through the model to collect activation statistics.
- **Step 2:** For each layer, quantize one column of the weight matrix at a time.
- **Step 3:** After quantizing a column, compute the quantization error and update the remaining (unquantized) weights in that layer to compensate.
- **Step 4:** Repeat for all columns, then move to the next layer.

Why it's popular

- Achieves near-FP16 accuracy at INT4 on most benchmarks
- Works on already-trained models — no retraining needed
- Supported by AutoGPTQ, ExLlama, and vLLM
- Standard choice for running large open-source models (LLaMA, Mistral) locally

Limitations

- Quantization process is slow (hours for large models)
 - Requires calibration data — quality of calibration data affects results
 - Can't easily be re-quantized with different settings once done
-

2.5 AWQ — Activation-Aware Weight Quantization

AWQ (2023, MIT) is another post-training quantization method, but takes a different approach to GPTQ. Instead of adjusting weights after quantization, AWQ identifies which weights are most important and protects them.

Key insight

Not all weights are equally important. A small subset of weights (those corresponding to large activation values — called salient weights) have a disproportionate impact on model output. Quantizing these poorly causes most of the accuracy loss.

How AWQ works

- **Step 1:** Run calibration data through the model and track activation magnitudes for each channel.
- **Step 2:** Identify salient channels — those with consistently large activations.
- **Step 3:** Scale up the weights in salient channels before quantization (making them easier to represent accurately in INT4) and scale down the activations to compensate.
- **Step 4:** Quantize all weights to INT4. The salient weights now have better quantization resolution because they were scaled up first.

AWQ vs GPTQ

AWQ: faster quantization process, generally better accuracy on generative tasks, hardware-friendly (no special kernel tricks needed)

GPTQ: more mature tooling, better supported across frameworks, slightly better on some perplexity benchmarks
In practice, AWQ tends to be preferred for deployment because it's faster to quantize and runs efficiently with the AWQ CUDA kernels (llm-awq, vLLM).

Interview Questions

Model Inferencing

Q1. What is a KV cache and why is it needed?

Answer: During autoregressive generation, the attention mechanism needs K and V matrices for all previous tokens at every step. Without caching, these are recomputed from scratch each time — $O(n^2)$ cost. KV cache stores these matrices after computing them once, so each new step only needs to compute K and V for the new token. This makes generation significantly faster. The cost is VRAM — KV cache grows linearly with sequence length.

Q2. Write the formula for KV cache memory size.

Answer: $\text{KV Cache} = 2 \times \text{num_layers} \times \text{num_heads} \times \text{head_dim} \times \text{seq_len} \times \text{bytes_per_element}$. The factor 2 is for K and V. For FP16, bytes_per_element = 2. A 13B model with 40 layers, 40 heads, head_dim 128, and 4096 context needs ~3.3 GB for KV cache alone.

Q3. What is continuous batching and how does it differ from static batching?

Answer: Static batching waits for all requests in a batch to finish before starting new ones — short requests waste GPU time waiting for long ones. Continuous batching (in-flight batching) slots in new requests the moment any existing request completes, so the GPU stays at near-100% utilization. Used in vLLM and TensorRT-LLM. Key metric improvement: throughput (requests/second) goes up substantially.

Q4. Explain speculative decoding. When does it help and when does it not?

Answer: A small draft model generates K tokens speculatively, then the large target model verifies all K tokens in a single parallel forward pass. Tokens are accepted up to the first disagreement. If tokens match, K tokens are generated for the cost of ~1 large model forward pass. Helps most for predictable tasks (code, structured output) where draft accuracy is high. Helps less for creative generation where the draft model often guesses wrong.

Q5. What is PagedAttention and why was it invented?

Answer: Standard KV cache pre-allocates memory based on max sequence length, causing fragmentation — GPU memory that's technically allocated but unused. PagedAttention (from vLLM) manages KV cache like virtual memory in an OS: allocates memory in fixed-size pages on demand, maps logical to physical pages. This eliminates fragmentation and allows much larger effective batch sizes on the same GPU.

Q6. What's the difference between tensor parallelism and pipeline parallelism?

Answer: Tensor parallelism splits individual weight matrices across GPUs — each GPU computes part of every layer. Requires high-bandwidth all-reduce communication on every forward pass but keeps all GPUs busy. Pipeline parallelism splits layers across GPUs — GPU1 does layers 1-10, GPU2 does 11-20 etc. Less communication but GPUs sit idle waiting for the previous stage (pipeline bubble). Tensor parallelism is better for low-latency; pipeline parallelism is better for throughput with high-bandwidth networks.

Quantization

Q7. What is quantization and why do we use it?

Answer: Quantization reduces the numeric precision of model weights from FP32/FP16 to INT8 or INT4. This shrinks model size (4x for INT8, 8x for INT4 vs FP32), speeds up inference (integer math is faster), and reduces VRAM usage so larger models fit on smaller GPUs. The trade-off is some accuracy loss, which different techniques try to minimize.

Q8. What's the difference between INT8 and INT4 quantization? When would you choose each?

Answer: INT8 uses 8-bit integers (-128 to 127) — moderate size reduction (~2x vs FP16) with minimal accuracy loss. Well-supported by hardware. Good for production where accuracy matters. INT4 uses 4-bit integers — more aggressive size reduction (~4x vs FP16) with higher accuracy risk. Usually needs group quantization and sometimes mixed precision to recover accuracy. Choose INT4 when running on memory-constrained hardware or edge devices where the trade-off is acceptable.

Q9. Explain how GPTQ works at a high level.

Answer: GPTQ quantizes models layer by layer using second-order information (Hessian). For each weight column in a layer, it quantizes the column and then updates the remaining unquantized weights to compensate for the introduced error. This layered error compensation is what makes GPTQ achieve near-FP16 accuracy at INT4. It requires a small calibration dataset and can take hours for large models, but the output can be used directly without retraining.

Q10. What makes AWQ different from GPTQ?

Answer: AWQ's key insight is that only a small fraction of weights matter a lot — those corresponding to large activation values (salient weights). Instead of compensating for error after quantization like GPTQ does, AWQ scales up salient weights before quantization so they get better representation in INT4, then scales down the activations to match. AWQ is faster to quantize and tends to perform better on generative tasks. GPTQ has more mature tooling. Both achieve strong INT4 results.

Q11. What is NF4 and where is it used?

Answer: NF4 (NormalFloat4) is a 4-bit data type designed specifically for neural network weights, which tend to follow a normal distribution. Instead of using evenly-spaced quantization levels like standard INT4, NF4 places more levels near zero (where most weights cluster) and fewer at extremes. This gives better precision where it's needed most. Used in QLoRA for quantizing base model weights before fine-tuning.

Q12. What is post-training quantization vs quantization-aware training?

Answer: Post-training quantization (PTQ) — quantize the model after training is complete, no retraining. Fast and requires only a small calibration dataset. Used by GPTQ and AWQ. Some accuracy loss. Quantization-aware training (QAT) — simulate quantization noise during training so the model learns to be robust to it. Better accuracy than PTQ but requires access to training data and compute for retraining. Usually worth it only when PTQ accuracy is insufficient.

Q13. A 70B model in FP16 won't fit on your 4xA100 (80GB each = 320GB total) setup. What are your options?

Answer:

- Quantize to INT8 or INT4 — 70B in FP16 is ~140 GB, in INT4 it's ~35 GB. Fits on a single A100 with room for KV cache.
- Use tensor parallelism to split the model across all 4 GPUs
- Use Flash Attention to reduce KV cache memory footprint
- Apply GPTQ or AWQ quantization for the best accuracy at INT4
- Combine: tensor parallelism + INT8 quantization

Q14. What is the quantization formula for converting a float to INT8?

Answer:

```
scale = (max_val - min_val) / 255
zero_point = round(-min_val / scale)
q = round(x / scale) + zero_point    # quantize
x_reconstructed = (q - zero_point) * scale  # dequantize
```

The quantization error is the difference between x and $x_{\text{reconstructed}}$. The goal of all quantization methods is to minimize this error.